In [ ]:
```
#Lab 2.4 KNN
```

In [ ]:
```
#Preparacion
```

In [2]:
```
from sklearn.neighbors import KNeighborsClassifier
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler

dataset = pd.read_csv('breast_cancer.csv')

dataset
```

Out[2]:

	mean.radius	mean.texture	mean.perimeter	mean.area	mean.smoothness	mean.com
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1203.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	
...	...	...	...	...	...	
564	21.56	22.39	142.00	1479.0	0.11100	
565	20.13	28.25	131.20	1261.0	0.09780	
566	16.60	28.08	108.30	858.1	0.08455	
567	20.60	29.33	140.10	1265.0	0.11780	
568	7.76	24.54	47.92	181.0	0.05263	

569 rows × 31 columns

In [3]:
```
dataset["cancer"].value_counts()
```

Out[3]:
```
cancer
1    357
0    212
Name: count, dtype: int64
```

In [5]:
```
num_y = dataset["cancer"].value_counts()

min_num = min(num_y)

min_class = num_y[num_y == min(num_y)].index[0]

undersamp = dataset[dataset["cancer"] != min_class].sample(min_num, random_state=10
```

```
dataset = pd.concat([undersamp, dataset[dataset["cancer"] == min_class]], axis=0)

dataset["cancer"].value_counts()
```

Out[5]:
```
cancer
1    212
0    212
Name: count, dtype: int64
```

In [ ]:

In [7]:
```
x_data = dataset.drop(['cancer'], axis=1)
y_data = dataset["cancer"]

scaler = MinMaxScaler()
scaler.fit(x_data)

x_data = scaler.transform(x_data)
```

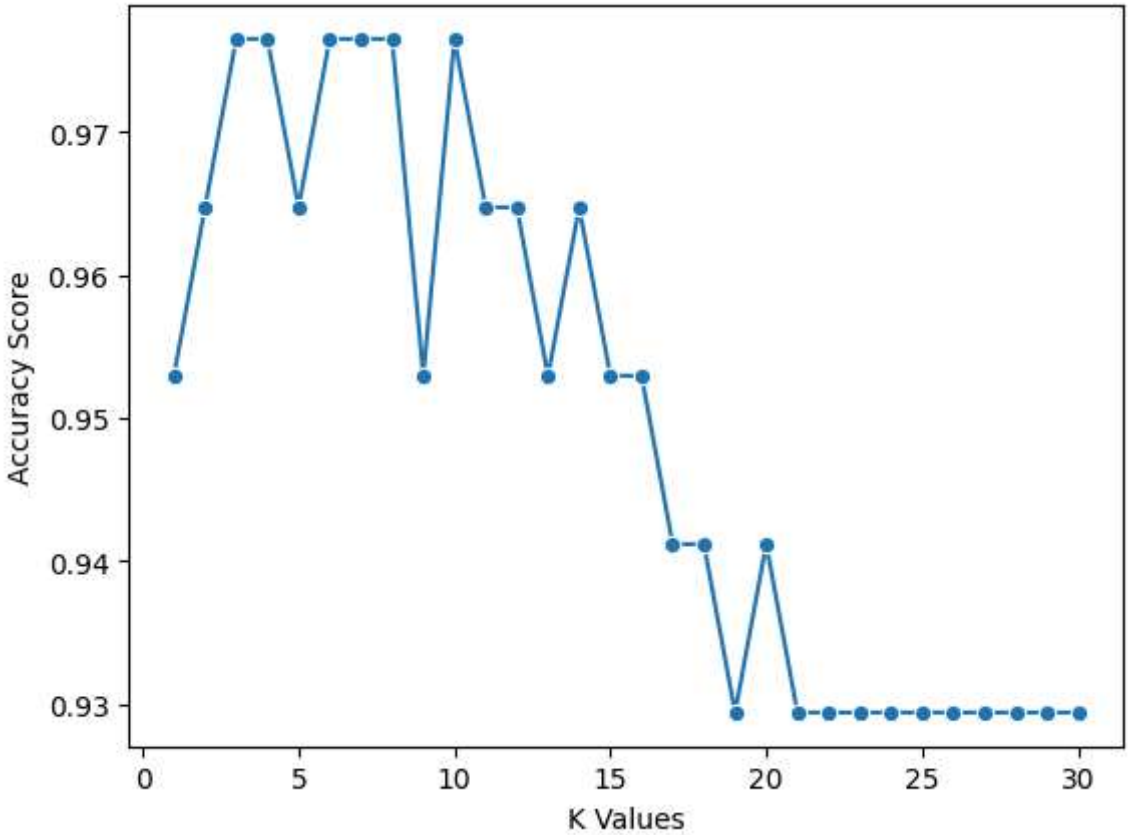
In [ ]:
```
#Separacion de datos
```

In [9]:
```
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
from sklearn.metrics import confusion_matrix
import seaborn as sns

x_train, x_test, y_train, y_test = train_test_split(
    x_data,
    y_data,
    train_size=0.8,
    random_state=14
)
```

In [ ]:
```
#Entrenamiento de modelo
```

In [10]:
```
from sklearn.metrics import accuracy_score

k_values = [i for i in range(1, 31)]
scores = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k, metric="manhattan")
    knn.fit(x_train, y_train)
    score = accuracy_score(y_test, knn.predict(x_test), normalize=True)
    scores.append(np.mean(score))

sns.lineplot(x=k_values, y=scores, marker='o')
plt.xlabel("K Values")
plt.ylabel("Accuracy Score")
```

Out[10]:
```
Text(0, 0.5, 'Accuracy Score')
```



In [11]:
```
classifier = KNeighborsClassifier(n_neighbors=5)
classifier.fit(x_train, y_train)

print(accuracy_score(y_train, classifier.predict(x_train), normalize=True))
print(accuracy_score(y_test, classifier.predict(x_test), normalize=True))
```

```
0.9646017699115044
0.9764705882352941
```

In [ ]:
```
#Evaluacion de resultados
```

In [15]:
```
!pip install mlxtend
```

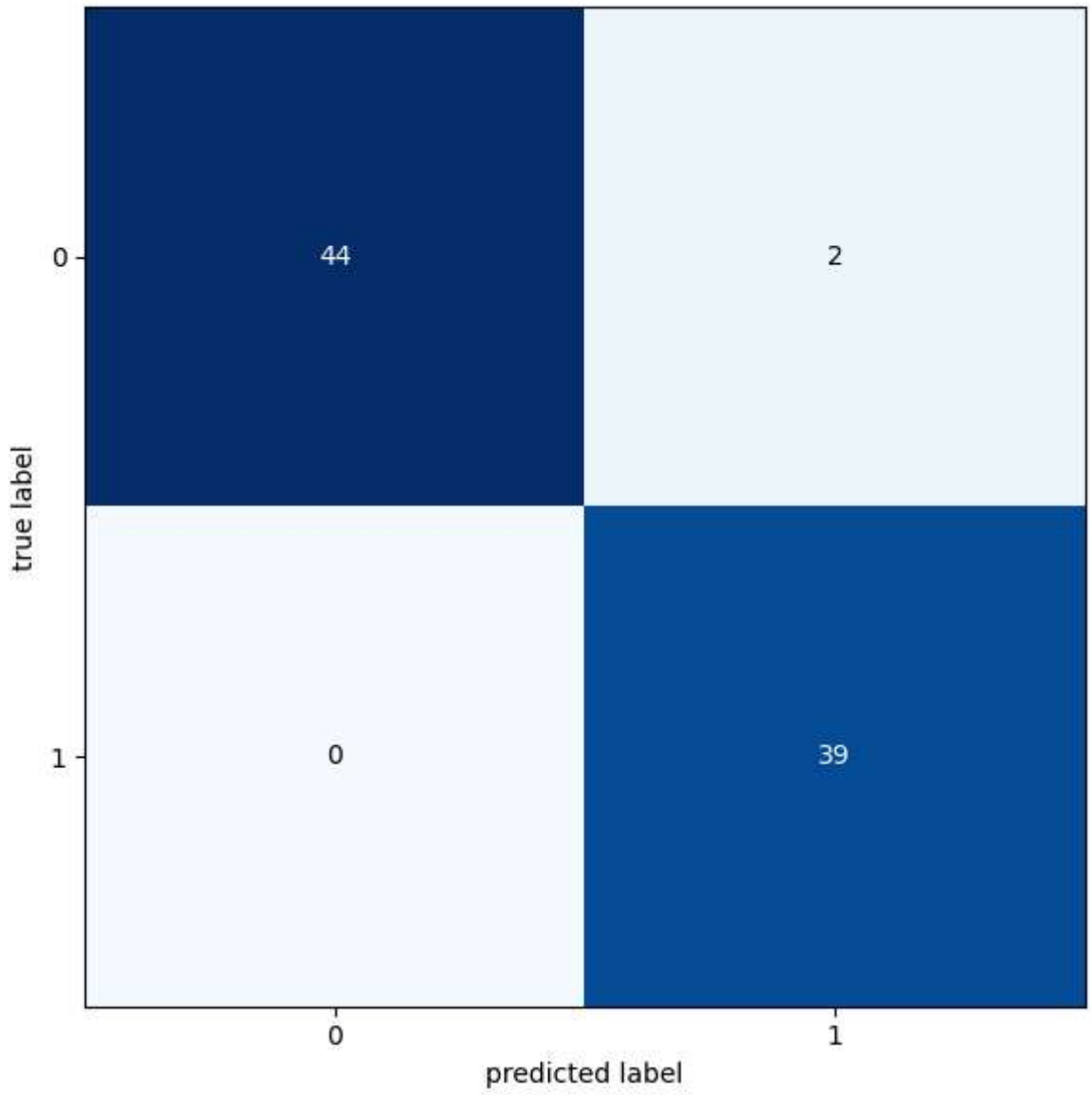
```
Requirement already satisfied: mlxtend in c:\users\jdele\anaconda3\lib\site-packages
(0.24.0)
Requirement already satisfied: scipy>=1.16.3 in c:\users\jdele\anaconda3\lib\site-pa
ckages (from mlxtend) (1.17.1)
Requirement already satisfied: numpy>=2.3.5 in c:\users\jdele\anaconda3\lib\site-pac
kages (from mlxtend) (2.3.5)
Requirement already satisfied: pandas>=2.3.3 in c:\users\jdele\anaconda3\lib\site-pa
ckages (from mlxtend) (3.0.1)
Requirement already satisfied: scikit-learn>=1.8.0 in c:\users\jdele\anaconda3\lib\s
ite-packages (from mlxtend) (1.8.0)
Requirement already satisfied: matplotlib>=3.10.8 in c:\users\jdele\anaconda3\lib\si
te-packages (from mlxtend) (3.10.8)
Requirement already satisfied: joblib>=1.5.2 in c:\users\jdele\anaconda3\lib\site-pa
ckages (from mlxtend) (1.5.3)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\jdele\anaconda3\lib\site
-packages (from matplotlib>=3.10.8->mlxtend) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\jdele\anaconda3\lib\site-pac
kages (from matplotlib>=3.10.8->mlxtend) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\jdele\anaconda3\lib\sit
e-packages (from matplotlib>=3.10.8->mlxtend) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\jdele\anaconda3\lib\sit
e-packages (from matplotlib>=3.10.8->mlxtend) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\users\jdele\anaconda3\lib\site-
packages (from matplotlib>=3.10.8->mlxtend) (25.0)
Requirement already satisfied: pillow>=8 in c:\users\jdele\anaconda3\lib\site-packag
es (from matplotlib>=3.10.8->mlxtend) (12.0.0)
Requirement already satisfied: pyparsing>=3 in c:\users\jdele\anaconda3\lib\site-pac
kages (from matplotlib>=3.10.8->mlxtend) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\jdele\anaconda3\lib
\site-packages (from matplotlib>=3.10.8->mlxtend) (2.9.0.post0)
Requirement already satisfied: tzdata in c:\users\jdele\anaconda3\lib\site-packages
(from pandas>=2.3.3->mlxtend) (2025.3)
Requirement already satisfied: six>=1.5 in c:\users\jdele\anaconda3\lib\site-package
s (from python-dateutil>=2.7->matplotlib>=3.10.8->mlxtend) (1.17.0)
Requirement already satisfied: threadpoolctl>=3.2.0 in c:\users\jdele\anaconda3\lib
\site-packages (from scikit-learn>=1.8.0->mlxtend) (3.6.0)
```

In [16]:

```python
from mlxtend.plotting import plot_confusion_matrix

y_pred = classifier.predict(x_test)

from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

cm = confusion_matrix(y_test, y_pred)

plot_confusion_matrix(conf_mat=cm, figsize=(6, 6), show_normed=False)
plt.tight_layout()

accuracy_score(y_test, y_pred)
```

Out[16]:

```
0.9764705882352941
```



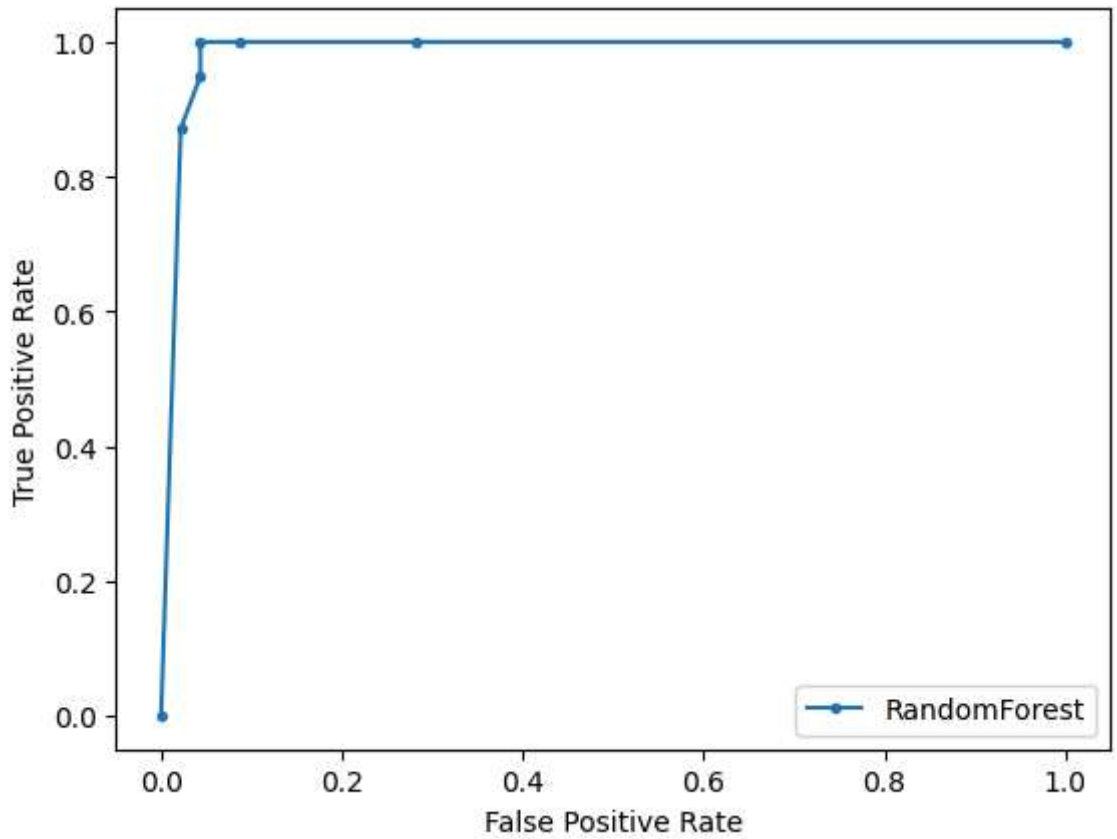
In [17]:
```
print(classification_report(y_test, round(pd.Series(classifier.predict(x_test)))))
```

```
              precision    recall  f1-score   support

           0       1.00      0.96      0.98        46
           1       0.95      1.00      0.97        39

    accuracy                           0.98        85
   macro avg       0.98      0.98      0.98        85
weighted avg       0.98      0.98      0.98        85
```

In [18]:
```
from sklearn.metrics import roc_auc_score, roc_curve
from matplotlib import pyplot

auc = roc_auc_score(y_test, classifier.predict_proba(x_test)[:, 1])
print('AUC: %.3f' % auc)

fpr, tpr, _ = roc_curve(y_test, classifier.predict_proba(x_test)[:, 1])

# plot the roc curve for the model
```

```
pyplot.plot(fpr, tpr, marker='.', label='RandomForest')

# axis labels
pyplot.xlabel('False Positive Rate')
pyplot.ylabel('True Positive Rate')

# show the legend
pyplot.legend()

# show the plot
pyplot.show()
```

```
AUC: 0.986
```



In [ ]: